PCCL

Fault tolerant collective communications over IP

Chapter 1: Why?

Motivation

- Perform all reduces over the Internet
- Tolerate and unwind *all* IO-induced failures
- Expose primitives to implement many different distributed optimization schemes (DDP, DiLoCo, streaming DiLoCo, …)
- Bit-wise identical results on all peers

What does PCCL enable?

- PCCL implements:
  Zero-Copy, bit-wise deterministic, interruptible, pipelined, quantized, asynchronous collective communications
- All peers obtain bit-identical all-reduce results
- PCCL's shared state is fully re-derivable from all performed all reduce results and an initial state.

Programming Paradigm

1) Connect to Master

```
PCCL_CHECK(pcclInit());

pcclComm_t *communicator{};
constexpr pcclCommCreateParams_t params{
    .master_address = {
        .inet = {.protocol = inetIPv4, .ipv4 = {.data[0]: 127, .data[1]: 0, .data[2]: 0, .data[3]: 1}},
        .port = CCOIP_PROTOCOL_PORT_MASTER
    },
    .peer_group = 0
};
PCCL_CHECK(pcclCreateCommunicator(&params, &communicator));
PCCL_CHECK(pcclConnect(communicator));
```

PCCL employs a dedicated master for its “micro-consensus scheme” & p2p connection information exchange.

Peers may only join when pre-existing set of peers periodically accept new peers into the run.

“`pcclConnect`” blocks until the peer is “accepted” into the run.

Programming Paradigm

2) Shared State

```
// Create shared state
pcclTensorInfo_t infos[2] = {
    {
        .name = "fc1.weight",
        .data = fc1_weights,
        .count = n_elements,
        .datatype = pcclFloat,
        .device_type = pcclDeviceCpu,
        .allow_content_inequality = false
    },
    {
        .name = "fc2.weight",
        .data = fc2_weights,
        .count = n_elements,
        .datatype = pcclFloat,
        .device_type = pcclDeviceCpu,
        .allow_content_inequality = false
    }
};
```

```
pcclSharedState_t shared_state{.revision = 0, .count = 2, .infos = infos};
```

- Reference all weight & optimizer state tensors from the shared state.
- Synchronized periodically.
- »No data transfer« occurs if content hashes and revision match!
- Application developers are expected to facilitate deterministic advancement of shared state per logical step such that shared state synchronization becomes a NO-OP.

Programming Paradigm

3) The main loop

A typical main loop consists of the following

- Accept new peers
- Topology-Optimization (bandwidth-aware)
- Synchronize Shared State
- Compute Work → *compute result*
- Collective Operations based on *compute result* → *comm result*
- Advance shared state with *comm result*

All non-async PCCL functions are strictly blocking and consensus-based.

- This means all peers call the same method at each point in time → “implicit barrier”
- Never must the application branch such that different peers call different micro-consensus-facilitating PCCL functions!

Programming Paradigm

3.1) The accept pump & topology optimization

```
size_t i = 0;
while (true) {
    i++;
    if (i > 1) {
        PCCL_CHECK(pcclUpdateTopology(communicator));
        PCCL_CHECK(pcclGetAttribute(communicator, attribute: PCCL_ATTRIBUTE_GLOBAL_WORLD_SIZE, &world_size));
    }

    if (world_size > 1) {
        PCCL_CHECK(pcclOptimizeTopology(communicator));
        PCCL_CHECK(pcclGetAttribute(communicator, attribute: PCCL_ATTRIBUTE_GLOBAL_WORLD_SIZE, &world_size));
    }

    if (world_size < 2) {
        std::this_thread::sleep_for(d: std::chrono::seconds(r: 1));
        continue;
    }
```

Peers can connect to the master any time.
They remain “registered” until they become “accepted”.
The pre-existing “accepted” peers all call “`pcclUpdateTopology`” to accept new peers into the run.
“`pcclOptimizeTopology`” will automatically perform bandwidth testing and form the optimal tour with an ATSP solver.

Programming Paradigm

3.2) Shared state Synchronization

```
pcclSharedStateSyncInfo_t sync_info{};
PCCL_CHECK(
    pcclSynchronizeSharedState(
        communicator, &shared_state,
        PCCL_SHARED_STATE_SYNC_STRATEGY_ENFORCE_POPULAR,
        &sync_info
    )
);
num_syncs++;
if (num_syncs > 1) {
    // Assert we never receive data. Only send.
    // Failure of this assert would indicate peer drift.
    assert(sync_info.rx_bytes == 0);
}
```

- During shared state synchronization, all shared state content is hashed using "*SimpleHash*", a fast, purpose built, GPU-optimized hashing function.
- If content mismatch occurs, re-transmission occurs to prevent peer-drift.
- A best-practice application should assert that after an initial sync on join, no further shared state reception occurs.

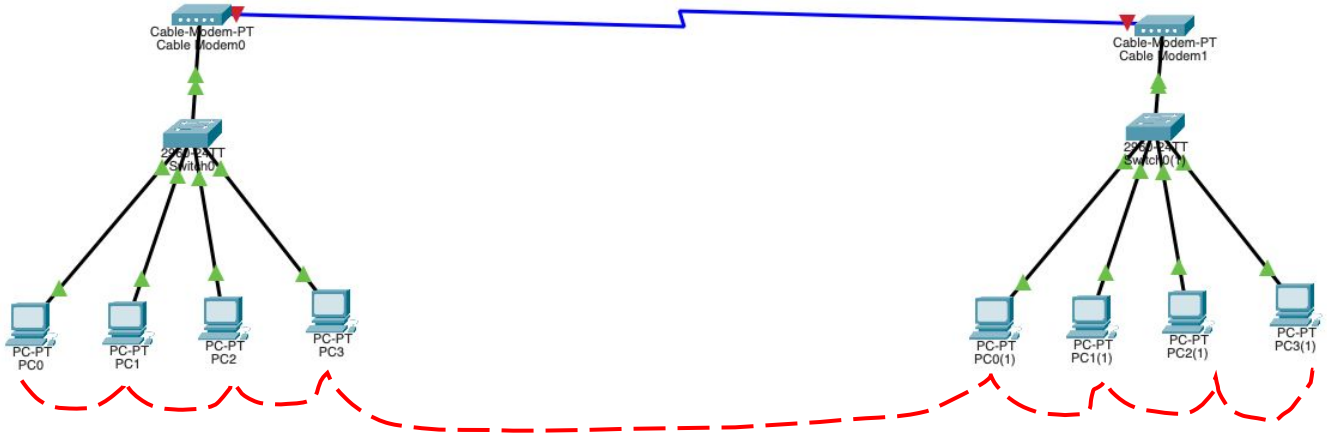
Programming Paradigm

3.3) Perform collective operations

```
do {
    constexpr pcclReduceDescriptor_t desc{
        .count = n_elements,
        .op = pcclSum,
        .tag = 0,
        .src_descriptor = {.datatype = pcclFloat, .distribution_hint = PCCL_DISTRIBUTION_HINT_NONE},
        .quantization_options = {.quantized_datatype = pcclUint8, .algorithm = pcclQuantMinMax},
    };
    pcclAllReduceAsync(gradients, gradients, &desc, communicator, &async_op);
    result = pcclAwaitAsyncReduce(&async_op, &reduce_info);
    pcclGetAttribute(communicator, attribute: PCCL_ATTRIBUTE_GLOBAL_WORLD_SIZE, &world_size);
    LOG(INFO) << "pcclAllReduce status " << result;
} while (result != pcclSuccess && world_size > 1);
```

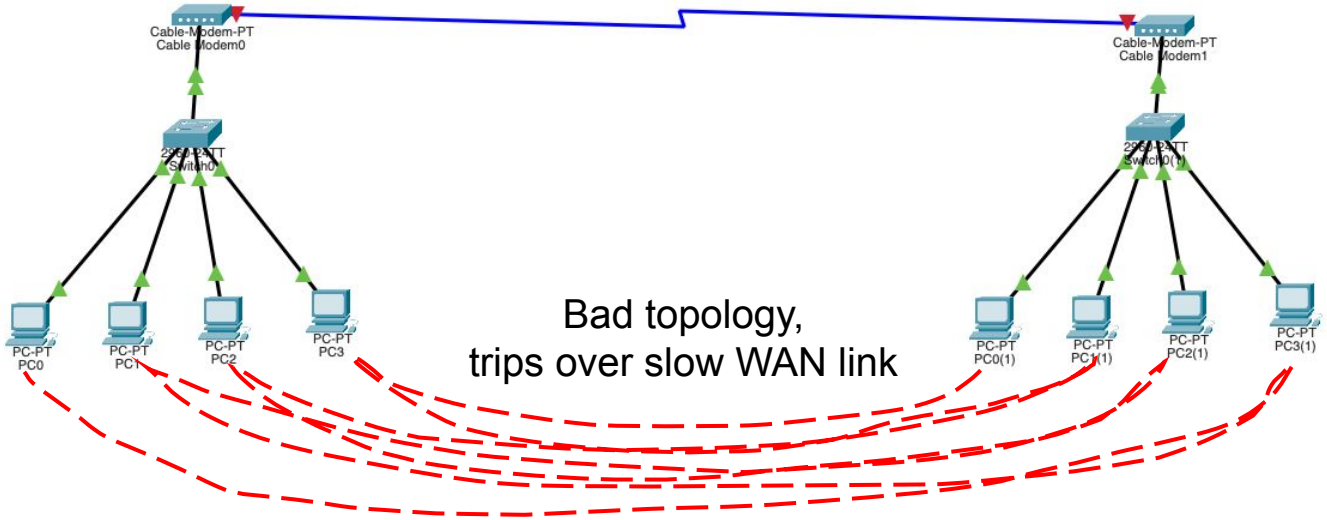
- Collective operations should be retried until a successful status is achieved.
- Quantization can be enabled.
- The world size can change after a collective operation has been awaited. It should be re-obtained.

Topology Optimization



Good topology!

Topology Optimization



Topology Optimization

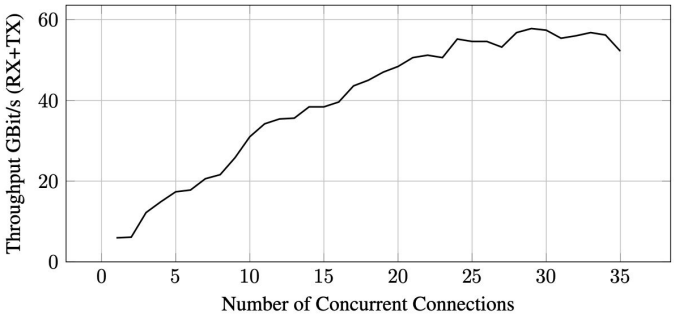
- Datacenters often have high speed Ethernet.
- Nodes often bind to public IPs directly.
- Packets addressed to computers in same datacenter can be delivered locally without bouncing the Gateway
- The discovered PCCL topology will reflect datacenter topologies of the participating peers
- PCCL will also take partial p2p unreachability into account
  - If peer A cannot ping peer B, the route will be constructed such that these peers do not have to talk with one another

Fault tolerance means fault tolerant

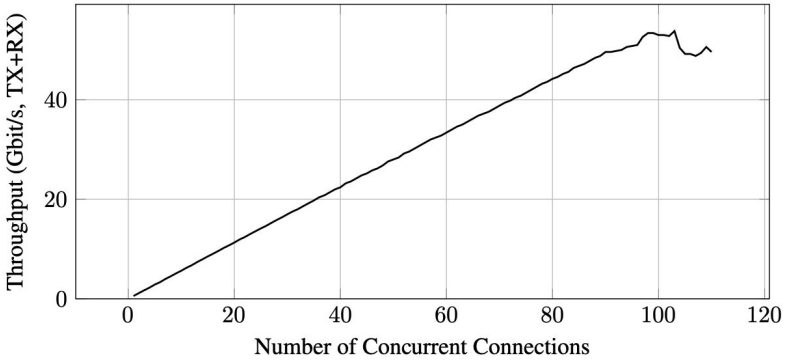
- Fault tolerance means no excuses
- All error paths must be handled correctly and gracefully
- Easy in theory, hard in practice

45 Gbit/s over the public internet

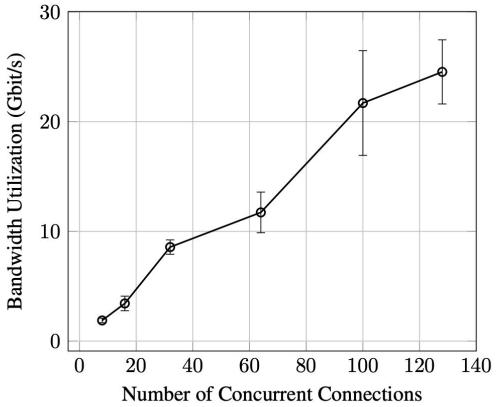
within EU



EU to US

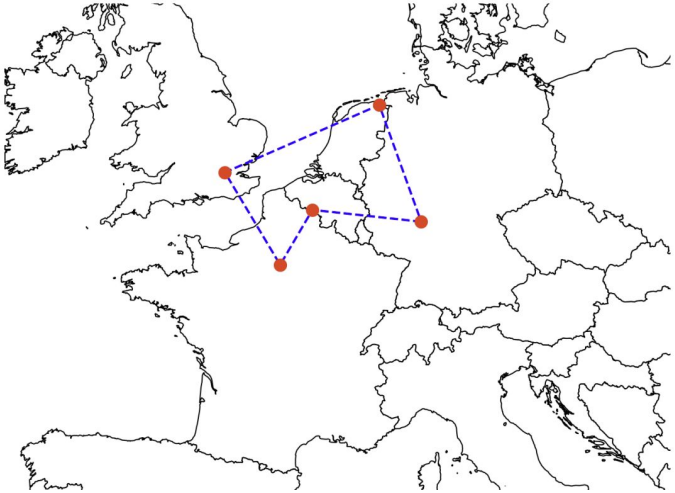


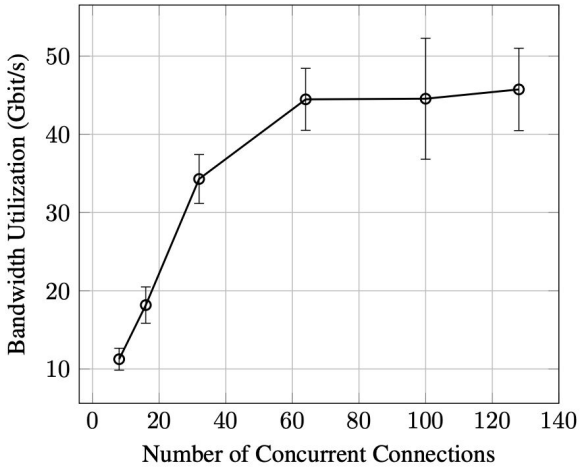
25 Gbit/s across US and EU



Metric	Value
Number of Peers	18
Reduce Contribution per Reduce Operation	64 MiB
Number of Reduce Operations	{ 8, 16, 32, 64, 100, 128 }
Max Number of Concurrent Reduce Operations	{ 8, 16, 32, 64, 100, 128 }
P2P Connection Pool Size	{ 8, 16, 32, 64, 100, 128 }

45 Gbit/s across EU-west





Metric	Value
Number of Peers	12
Reduce Contribution per Reduce Operation	64 MiB
Number of Reduce Operations	{ 8, 16, 32, 64, 128 }
Max Number of Concurrent Reduce Operations	{ 8, 16, 32, 64, 128 }
P2P Connection Pool Size	{ 8, 16, 32, 64, 128 }

Fault tolerance Demo

Chapter 2: Agony

Socket Implementation Quirks #1

Closing causes data loss

```
uint16_t packet_id = read_uint16(socket_fd);
if (!valid_packet_ids.contains(packet_id)) {
    write(socket_fd, "Error: bad data", …); // !!!
    close(socket_fd); // causes a RST
     return;
}
```

- Fix: graceful shutdown using SO_LINGER

Socket Implementation Quirks #2

Close a socket read by a different thread

```
void run() {
    // Thread 2
    while (true) {
        uint8_t data[255];
        if (recv(socket_fd, data, sizeof(data), 0) != 0) { // STUCK!
            break;
        }
    }
}
void interrupt() {
    close(socket_fd);
}
```

- Fix: shutdown(RDWR)

How to actually close a socket

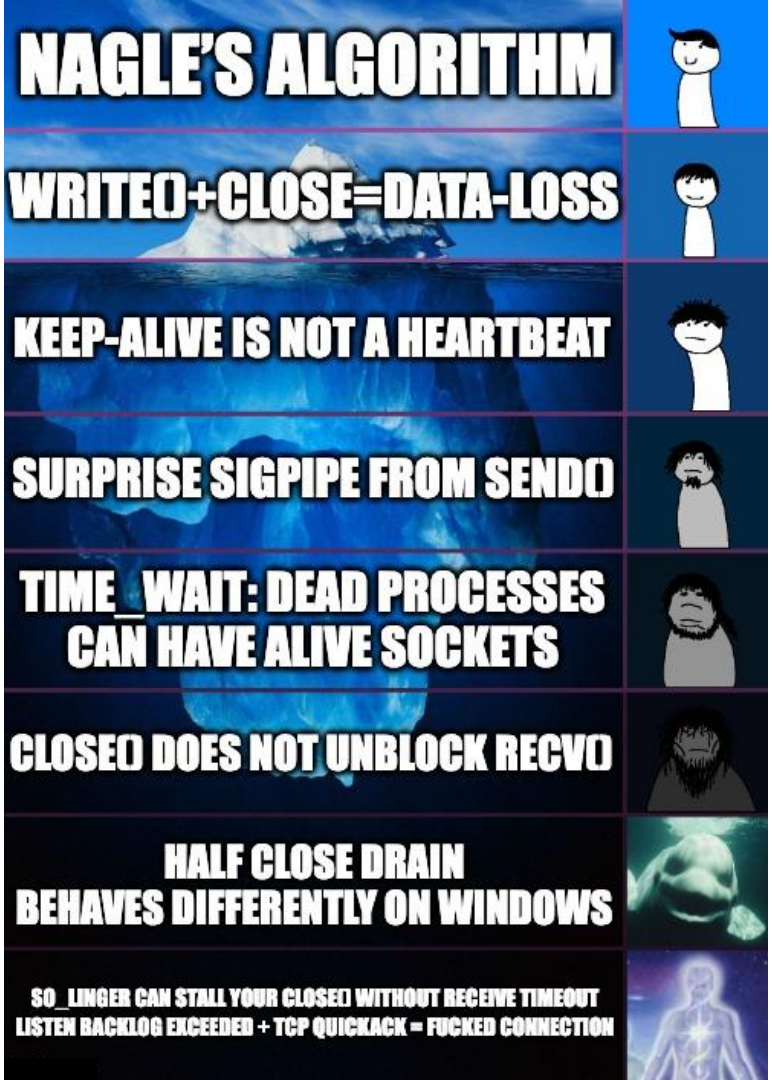
```
bool tinysockets::BlockingIOSocket::closeConnection(const bool allow_data_discard) {
    if (socket_fd == 0) {
        return false;
    }

    timeval tv{};
    tv.tv_sec = 1;
    tv.tv_usec = 0;
    setsockoptvp(socket_fd, level: SOL_SOCKET, optname: SO_RCVTIMEO, &tv, optlen: sizeof(tv));

    if (!allow_data_discard) {
        // linger on close to ensure all data is sent and at least acked by the peer's kernel
        linger l{};
        l.l_onoff = 1;
        l.l_linger = 5;
        setsockoptvp(socket_fd, level: SOL_SOCKET, optname: SO_LINGER, &l, optlen: sizeof(l));
    } else {
        // set SO_LINGER to 0 to force a hard close
        linger l{};
        l.l_onoff = 1;
        l.l_linger = 0;
        setsockoptvp(socket_fd, level: SOL_SOCKET, optname: SO_LINGER, &l, optlen: sizeof(l));
    }

    // Now shut everything down.
    // This is needed to ensure recv() unblock and return an error.
    // Docker is pedantic about this.
    shutdown(socket_fd, SHUT_RDWR);

    // finally, close the socket
    closesocket(socket_fd);
    socket_fd = 0;
    return true;
}
```


NAGLE'S ALGORITHM
WRITE()+CLOSE=DATA-LOSS
KEEP-ALIVE IS NOT A HEARTBEAT
SURPRISE SIGPIPE FROM SEND()
TIME_WAIT: DEAD PROCESSES
CAN HAVE ALIVE SOCKETS
CLOSE() DOES NOT UNBLOCK RECV()
HALF CLOSE DRAIN
BEHAVES DIFFERENTLY ON WINDOWS
SO_LINGER CAN STALL YOUR CLOSE() WITHOUT RECEIVE TIMEOUT
LISTEN BACKLOG EXCEEDED + TCP QUICKACK = FUCKED CONNECTION

Still not enough for fault tolerance

- Any error must be properly and cleanly unwound to the user
- Clean tear down of application state in response to errors
  - Never must errors produce “dirty” application state!
- Often involves complicated recovery procedures
- E.g. IO failures in some peers are not apparent to all peers
  - Involves coordination with the master to even discover the failure state
- Cross-Connection packet handling chains → potential footgun
  - No guarantee of cross connection packet arrival
  - “Spooky seemingly unsolicited packets”
    - Micro-consensus scheme “guarding” state transitions

Isn’t this trivial?

- “just handle all the errors bro”
  - Yesn’t.
- Easy in theory, hard in practice.
- Too permissive systems *cannot* even *theoretically* be fault tolerant
  - Restrict legal state transitions in your system as much as possible.
  - Do not imply promises with your API surface you cannot keep!
    - It happens quicker than you may think!

Deterministic advancement of shared state

- PCCL guarantees bit-identical results of all reduce operations
- Allows for deterministic advancement of model weights with optimizer step such that hashes match *exactly*
  - Elementwise operations do not induce GPU-indeterminism
  - Neither do matmuls, nor flash attention forward and many more
    - (given same block sizes, and tensor core generation)
  - Thus even Muon's Newton-Schulz iterations are unproblematic
- Does Nvidia guarantee this?
  - Absolutely not.
- However…

__nv_expf from the 980 Ti to the B200

- We test __nv_expf implemented in libdevice.bc across GPU generations and find identical behavior

```
Results over all 4294967296 float bit-patterns:
     NaN outputs: 16777214
     Inf outputs: 1020169705
     Finite outputs: 3258020377
     Sum check: 4294967296
Output-bits hash:
     Sum of bits: 4602279786742895247
     XOR of bits: 0x45ABDA35
```

GPUs tested:

- GTX 980 Ti (sm_52)
- GTX 1060 (sm_61)
- RTX 4090 (sm_89)
- GH 200 (sm_90)
- B200 (sm_100)

Fully deterministic MNIST training on GPU

```
(base) mike@cuda-container-01:~/CLionProjects/verifymnist$ c
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
Epoch 10 weight hash: 0x8406dbb808a69466
```

```
Terminal   mike@cuda-cont...cts/verifymnist   mike@cuda-cont...cts/verifymnist (2)
Sat Jul 12 14:28:18 2025                    (Press h for help or q to quit)

NVITOP 1.4.2        Driver Version: 565.57.01        CUDA Driver Version: 12.7

GPU Fan Temp Perf Pwr:Usg/Cap          Memory-Usage   GPU-Util  Compute M.
  0 30%  48C  P0  131W / 450W   39.23GiB / 47.99GiB     99%      Default   MEM: 81.7%   UTL: 99%
  1 30%  36C  P8   25W / 450W   19.69MiB / 47.99GiB      0%      Default   MEM: | 0.0%   UTL: | 0%

[ CPU:                                                                                MAX ]  ( Load Average: 175.2 163.8 70.77 )
[ MEM:                                                                                       USED: 51.51GiB ]  [ SWP: | 0.0%  ]
                                  21.7%

Processes:                                                                                                   mike@cuda-container-01
GPU     PID      USER  GPU-MEM %SM  %CPU  %MEM     TIME  COMMAND

  0 1435077 C    mike 396.0MiB  0  33.5   0.2     0:35  ./train_mnist
  0 1435078 C    mike 396.0MiB  0  30.8   0.2     0:35  ./train_mnist
  0 1435079 C    mike 396.0MiB  0  27.8   0.2     0:35  ./train_mnist
  0 1435080 C    mike 396.0MiB  0  38.4   0.2     0:35  ./train_mnist
  0 1435081 C    mike 396.0MiB  0  29.4   0.2     0:36  ./train_mnist
  0 1435082 C    mike 396.0MiB  0  28.8   0.2     0:35  ./train_mnist
  0 1435083 C    mike 396.0MiB  0  34.9   0.2     0:36  ./train_mnist
  0 1435084 C    mike 396.0MiB  0  29.7   0.2     0:35  ./train_mnist
  0 1435085 C    mike 396.0MiB  0  41.1   0.2     0:35  ./train_mnist
  0 1435086 C    mike 396.0MiB  0  28.5   0.2     0:35  ./train_mnist
  0 1435087 C    mike 396.0MiB  0  31.2   0.2     0:35  ./train_mnist
  0 1435088 C    mike 396.0MiB  0  30.2   0.2     0:35  ./train_mnist
  0 1435089 C    mike 396.0MiB  0  31.2   0.2     0:35  ./train_mnist
  0 1435090 C    mike 396.0MiB  0  33.1   0.2     0:36  ./train_mnist
  0 1435091 C    mike 396.0MiB  0  29.0   0.2     0:35  ./train_mnist
  0 1435092 C    mike 396.0MiB  0  34.5   0.2     0:36  ./train_mnist
  0 1435093 C    mike 396.0MiB  0  31.3   0.2     0:36  ./train_mnist
  0 1435094 C    mike 396.0MiB  0  32.6   0.2     0:36  ./train_mnist
  0 1435095 C    mike 396.0MiB  0  30.5   0.2     0:36  ./train_mnist
  0 1435096 C    mike 396.0MiB  0  30.6   0.2     0:35  ./train_mnist
  0 1435097 C    mike 396.0MiB  0  32.6   0.2     0:35  ./train_mnist
  0 1435098 C    mike 396.0MiB  0  31.1   0.2     0:35  ./train_mnist
```

Any Questions?